

Kali Linux con IA: Guía para Claude Desktop en Windows 11 + MCP en Kali Linux

(Mayo 2026)

Se basa en:

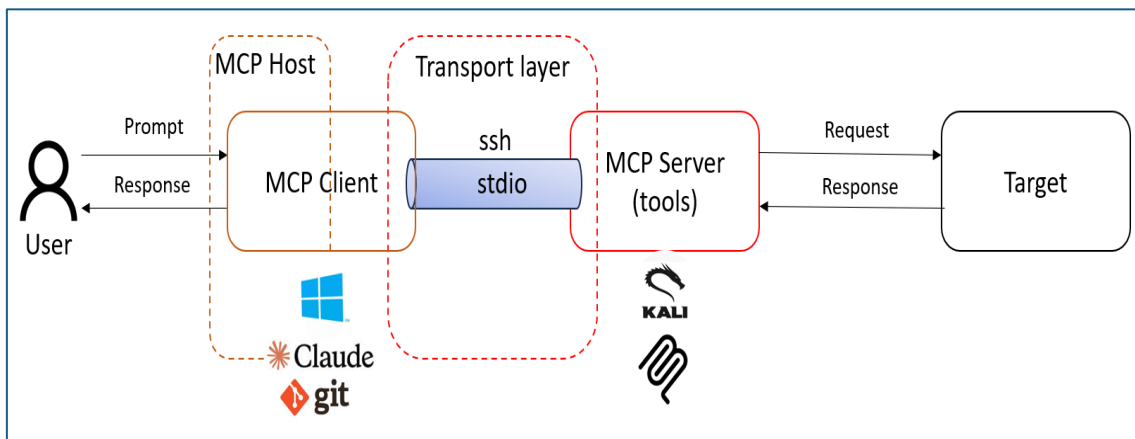
- La guía oficial de [Kali para macOS](#)
- El vídeo de YouTube [Kali Linux con IA](#)
- El paquete oficial mcp-kali-server de Kali Tools

Requisitos previos

- **Kali Linux** instalado y en ejecución (máquina virtual, cloud o bare-metal). Debe estar en la misma red que tu Windows 11 y accesible por IP.
- **Windows 11** actualizado.
- Usuario por defecto en Kali: kali (si usas otro, cámbialo en los comandos).
- Conexión de red estable (sin firewall bloqueando SSH puerto 22).
- Cuenta en Anthropic para Claude Desktop.

Advertencia de seguridad:

- Nunca expongas el servidor MCP a internet directamente.
- Usa claves SSH sin passphrase (como se indica).
- Claude pedirá confirmación antes de ejecutar cada comando en Kali (buena práctica de seguridad).



1. Configuración en Kali Linux

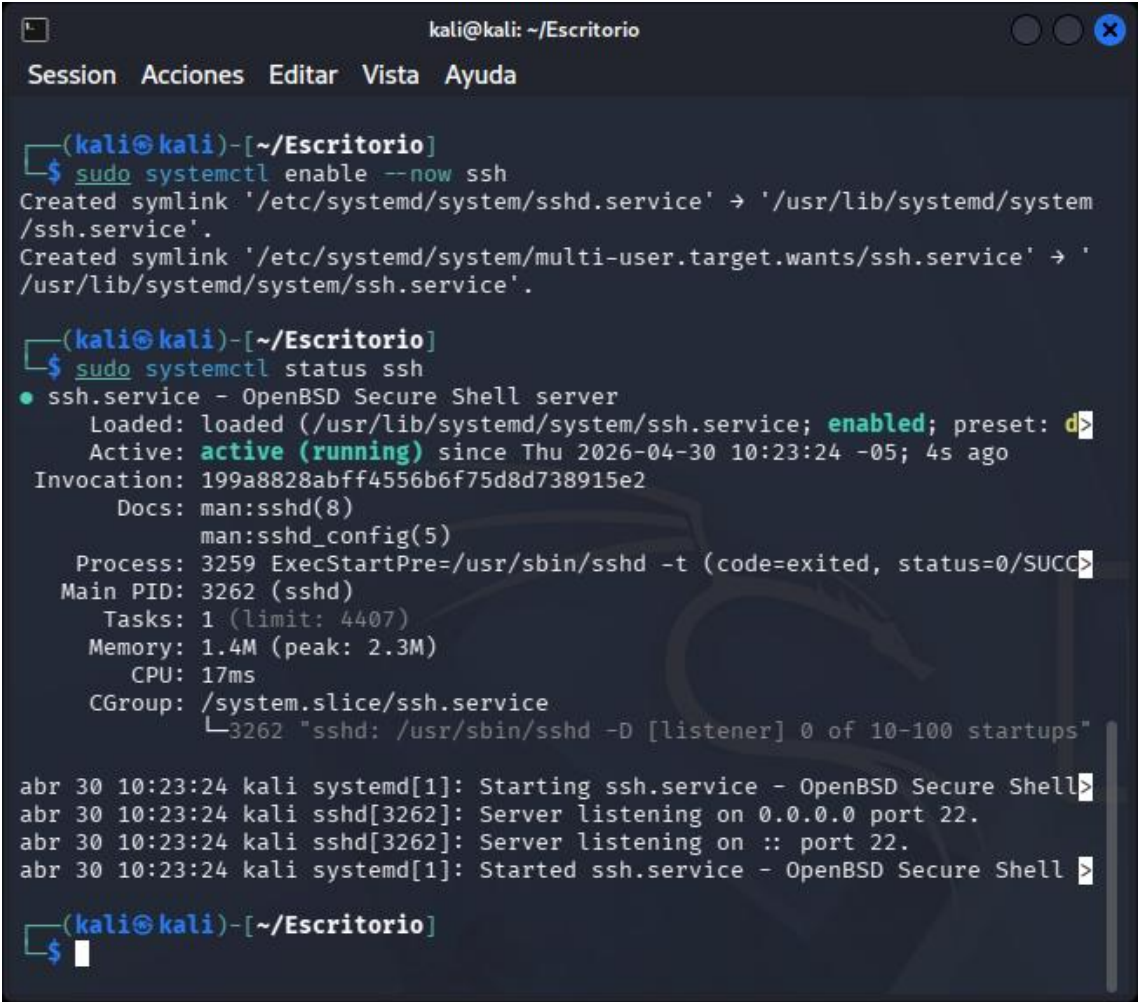
1.1. Habilitar SSH (si no está configurado)

Abre una terminal en Kali:

```
$ sudo apt update  
$ sudo apt install -y openssh-server  
$ sudo systemctl enable --now ssh
```

Verifica que SSH esté escuchando:

```
$ sudo systemctl status ssh
```



```
kali@kali: ~/Escritorio  
Session Acciones Editar Vista Ayuda  
  
(kali@kali)-[~/Escritorio]  
$ sudo systemctl enable --now ssh  
Created symlink '/etc/systemd/system/ssh.service' → '/usr/lib/systemd/system/ssh.service'.  
Created symlink '/etc/systemd/system/multi-user.target.wants/ssh.service' → '/usr/lib/systemd/system/ssh.service'.  
  
(kali@kali)-[~/Escritorio]  
$ sudo systemctl status ssh  
● ssh.service - OpenBSD Secure Shell server  
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: d)  
   Active: active (running) since Thu 2026-04-30 10:23:24 -05; 4s ago  
 Invocation: 199a8828abff4556b6f75d8d738915e2  
    Docs: man:sshd(8)  
          man:sshd_config(5)  
 Process: 3259 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)  
 Main PID: 3262 (sshd)  
   Tasks: 1 (limit: 4407)  
  Memory: 1.4M (peak: 2.3M)  
    CPU: 17ms  
   CGroup: /system.slice/ssh.service  
           └─3262 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"  
  
abr 30 10:23:24 kali systemd[1]: Starting ssh.service - OpenBSD Secure Shell  
abr 30 10:23:24 kali sshd[3262]: Server listening on 0.0.0.0 port 22.  
abr 30 10:23:24 kali sshd[3262]: Server listening on :: port 22.  
abr 30 10:23:24 kali systemd[1]: Started ssh.service - OpenBSD Secure Shell  
  
(kali@kali)-[~/Escritorio]  
$
```

1.2. Instalar y configurar el servidor MCP (mcp-kali-server)

```
$ sudo apt update  
$ sudo apt install -y mcp-kali-server
```

Iniciar el servidor MCP (esto crea la API en http://127.0.0.1:5000):

```
kali-server-mcp
```

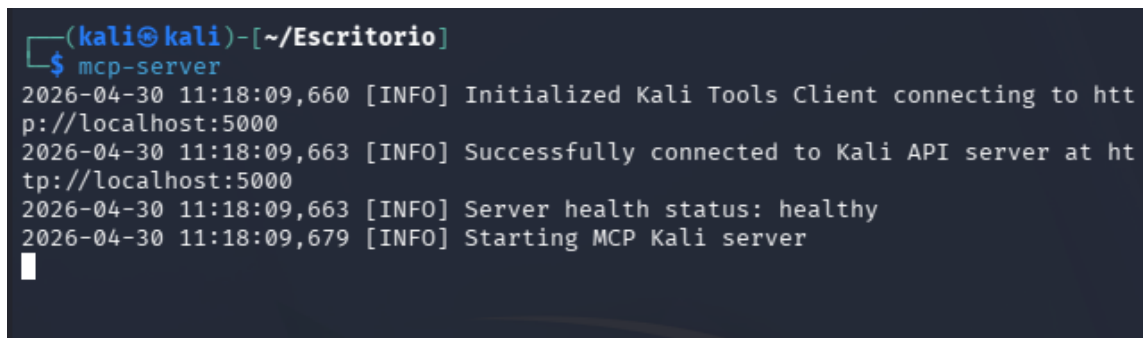
Salida esperada (ejemplo):

```
2026-... [INFO] Starting Kali Linux Tools API Server on 127.0.0.1:5000
* Running on http://127.0.0.1:5000
```

Prueba

Para comprobar que todo funciona hasta ahora, en **otro terminal ejecuta mcp-server** (esto es lo que nuestro cliente MCP, Claude Desktop, acabará ejecutando):

```
$ mcp-server
```

A terminal window with a dark background. The prompt is (kali@kali)-[~/Escritorio]. The command mcp-server has been entered. The output shows four lines of log messages: '2026-04-30 11:18:09,660 [INFO] Initialized Kali Tools Client connecting to http://localhost:5000', '2026-04-30 11:18:09,663 [INFO] Successfully connected to Kali API server at http://localhost:5000', '2026-04-30 11:18:09,663 [INFO] Server health status: healthy', and '2026-04-30 11:18:09,679 [INFO] Starting MCP Kali server'. A cursor is visible on the line following the last message.

```
(kali@kali)-[~/Escritorio]
$ mcp-server
2026-04-30 11:18:09,660 [INFO] Initialized Kali Tools Client connecting to http://localhost:5000
2026-04-30 11:18:09,663 [INFO] Successfully connected to Kali API server at http://localhost:5000
2026-04-30 11:18:09,663 [INFO] Server health status: healthy
2026-04-30 11:18:09,679 [INFO] Starting MCP Kali server
█
```

BUG: health check falla

En el momento de hacer este manual, existe un bug en el MCP de Kali Linux que es necesario corregir, específicamente en el archivo `mcp_server.py` ubicado en `/usr/share/mcp-kali-server`, su función de verificación no encontraba las herramientas correctamente (posiblemente busca en un PATH distinto), pero `execute_command` sí tiene acceso al entorno completo donde `nmap` está instalado en `/usr/bin/nmap`.

Solución:

Usar shutil.which directamente (más limpio y eficiente):

ANTES (buggy):

```
# Health check endpoint
@app.route("/health", methods=["GET"])
def health_check():
    """Health check endpoint."""
    # Check if essential tools are installed
    essential_tools = ["nmap", "gobuster", "dirb", "nikto"]
    tools_status = {}

    for tool in essential_tools:
        try:
            result = execute_command(["which", tool])
            tools_status[tool] = result["success"]
        except:
            tools_status[tool] = False

    all_essential_tools_available = all(tools_status.values())
```

DESPUÉS (fix):

```
import shutil
.
.
.
# Health check endpoint
@app.route("/health", methods=["GET"])
def health_check():
    """Health check endpoint."""
    # Check if essential tools are installed
    essential_tools = ["nmap", "gobuster", "dirb", "nikto"]
    tools_status = {}

    for tool in essential_tools:
        tools_status[tool] = shutil.which(tool) is not None

    all_essential_tools_available = all(tools_status.values())
```

1.3. Instalar herramientas de Kali que usará MCP

En otra terminal (o después de ^C en la anterior):

```
$ sudo apt install -y dirb gobuster nikto nmap enum4linux-ng hydra
john metasploit-framework sqlmap wpscan wordlists
```

```
$ sudo gunzip -v /usr/share/wordlists/rockyou.txt.gz
```

Saldrá un aviso de herramientas faltantes la primera vez; esto lo soluciona.

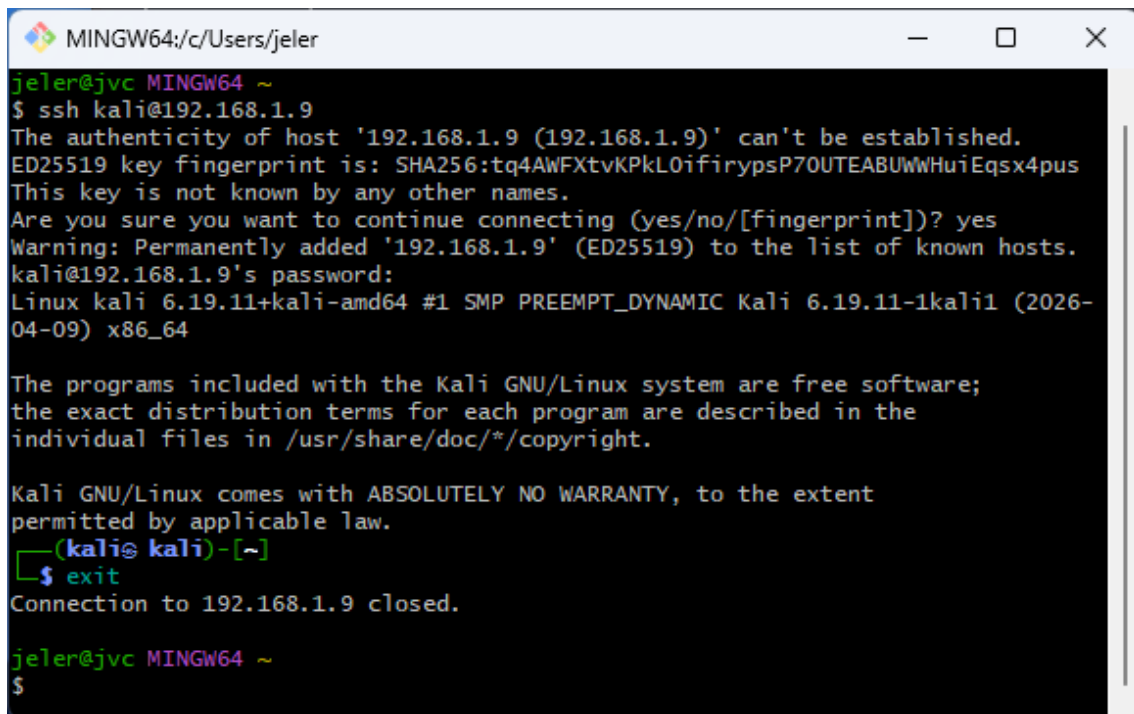
2. Configuración en Windows 11

2.1. Instalar Git for Windows (necesario para ssh.exe)

1. Descarga desde <https://git-scm.com/download/win>
2. Instala con opciones predeterminadas (importante: **agrega Git a PATH**).

2.2. Instalar Claude Desktop

1. Ve a <https://claude.com/download> y descarga la versión para Windows (última disponible en marzo 2026).
2. Instala y completa el primer inicio de sesión con tu cuenta Anthropic.



```
MINGW64:/c/Users/jeler
jeler@jvc MINGW64 ~
$ ssh kali@192.168.1.9
The authenticity of host '192.168.1.9 (192.168.1.9)' can't be established.
ED25519 key fingerprint is: SHA256:tq4AWFXtvKPkL0ifirypsP70UTEABUWWHuiEqsx4pus
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.1.9' (ED25519) to the list of known hosts.
kali@192.168.1.9's password:
Linux kali 6.19.11+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.19.11-1kali1 (2026-04-09) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
(kali@kali)-[~]
└─$ exit
Connection to 192.168.1.9 closed.

jeler@jvc MINGW64 ~
$
```

2.3. Generar llaves SSH sin passphrase (en Git Bash)

Abre **Git Bash** como usuario normal:

```
$ cd ~/.ssh

$ ssh-keygen.exe -t ed25519 -C "claude@kali"

# Nombre del archivo: llave1 (presiona Enter)

# Passphrase: deja vacío (presiona Enter dos veces)
```

Salida: se crean las llaves **llave1** (privada) y **llave1.pub** (pública).

```
MINGW64:/c/Users/jeler/.ssh
jeler@jvc MINGW64 ~/.ssh
$ ssh-keygen.exe -t ed25519 -C "Claude@kali"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/c/Users/jeler/.ssh/id_ed25519): llave1
Enter passphrase for "llave1" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in llave1
Your public key has been saved in llave1.pub
The key fingerprint is:
SHA256:SuwskpBrNS177/sXGR4xWpmI8hIyH5m07KNTC20g/PY Claude@kali
The key's randomart image is:
+--[ED25519 256]--+
|    ..o . . o    |
|   o.*.. . *    |
|  . +o=  o o    |
|o.. o+ . . o    |
|+. B =+ S . +   |
| o+.O+o.  +     |
|..o*.o+      .  |
| . ..+..      .  |
|    E.+o..      |
+-----[SHA256]-----+

jeler@jvc MINGW64 ~/.ssh
$
```

2.4. Copiar la clave pública a Kali

En Git Bash:

```
$ ssh-copy-id -i ~/.ssh/llave1.pub kali@TU_IP_KALI
```

(reemplaza TU_IP_KALI por la IP real de tu Kali, ej. 192.168.1.9.

Si pide contraseña la primera vez, introdúcela. Luego ya no la necesitará.

```
MINGW64:/c/Users/jeler/.ssh
jeler@jvc MINGW64 ~/.ssh
$ ssh-copy-id -i ~/.ssh/llave1.pub kali@192.168.1.9
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/c/Users/jeler/.ssh/llave1.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
kali@192.168.1.9's password:

Number of key(s) added: 1

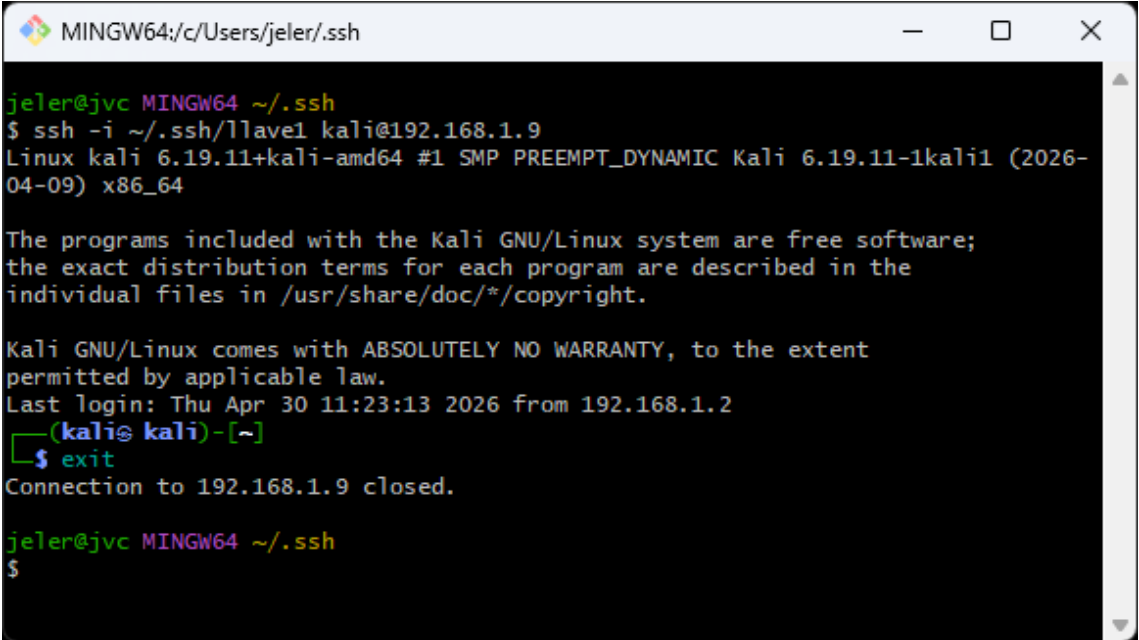
Now try logging into the machine, with: "ssh -i /c/Users/jeler/.ssh/llave1 'kali@192.168.1.9'"
and check to make sure that only the key(s) you wanted were added.

jeler@jvc MINGW64 ~/.ssh
$
```

Prueba de conexión sin contraseña:

```
$ ssh -i ~/.ssh/llave1 kali@TU_IP_KALI
```

Debe entrar directamente. Escribe `exit` para salir.



```
MINGW64:/c/Users/jeler/.ssh

jeler@jvc MINGW64 ~/.ssh
$ ssh -i ~/.ssh/llave1 kali@192.168.1.9
Linux kali 6.19.11+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.19.11-1kali1 (2026-04-09) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Apr 30 11:23:13 2026 from 192.168.1.2
└─(kali@kali)-[~]
└─$ exit
Connection to 192.168.1.9 closed.

jeler@jvc MINGW64 ~/.ssh
$
```

2.5. Configurar el cliente MCP en Claude Desktop

1. Abre **Claude Desktop**.
2. Ve a **Claude → Ajustes → Desarrollador → Editar configuración**. (Se abrirá automáticamente `claude_desktop_config.json` en la carpeta correcta, normalmente:
`C:\Users\TU_USUARIO\AppData\Local\Packages\Claude_...\LocalCache\Roaming\Claude o %APPDATA%\Claude\`).

3. **Reemplaza todo el contenido con esto** (ajusta las rutas e IP):

```
{
  "preferences": {
    "coworkWebSearchEnabled": true,
    "coworkScheduledTasksEnabled": false,
    "ccdScheduledTasksEnabled": false
  },
  "mcpServers": {
    "mcp-kali-server": {
      "command": "C:\\Program Files\\Git\\usr\\bin\\ssh.exe",
      "args": [
        "-i", "C:\\Users\\jeler\\.ssh\\llave1",
        "-o", "StrictHostKeyChecking=no",
        "kali@192.168.1.9",
        "mcp-server"
      ]
    }
  }
}
```



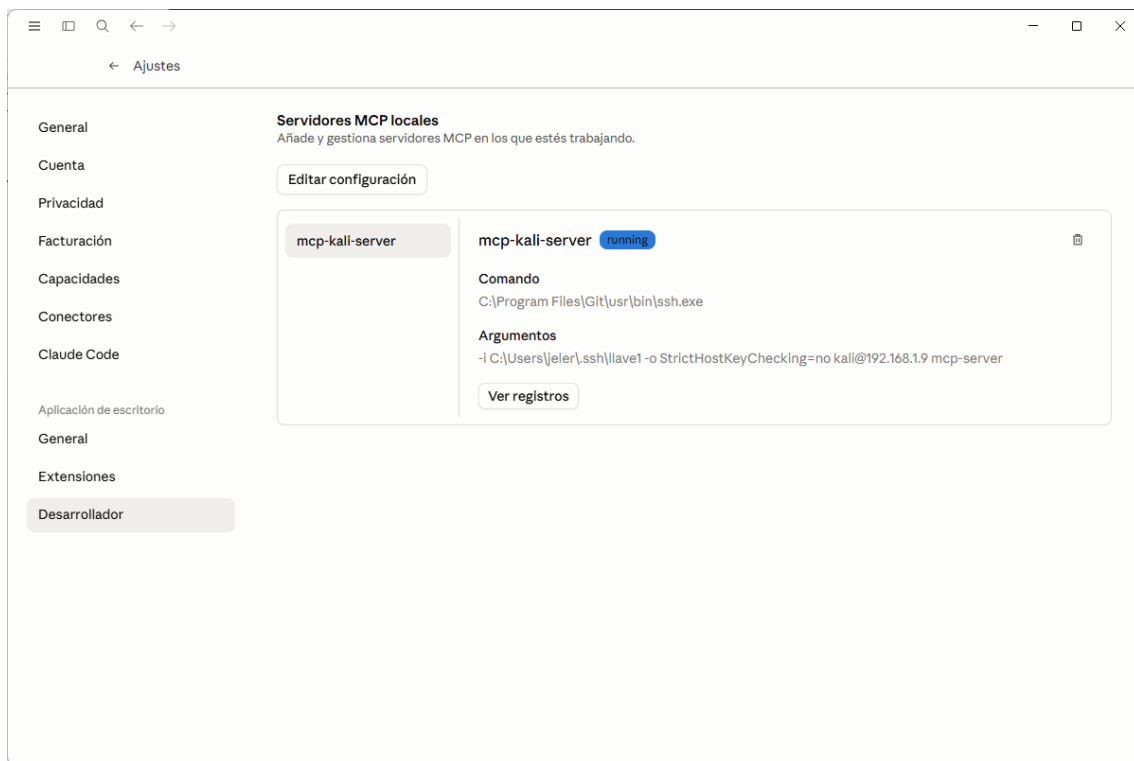
```

    ],
    "transport": "stdio"
  }
}

```

Notas importantes:

- Usa **doble barra** \ en las rutas de Windows.
 - Reemplaza TU_USUARIO por tu nombre de usuario de Windows (ej. jeler).
 - Reemplaza TU_IP_KALI por la IP real de Kali.
 - Guarda el archivo.
4. **Cierra completamente Claude Desktop** y vuélvelo a abrir (la configuración se carga solo al inicio).



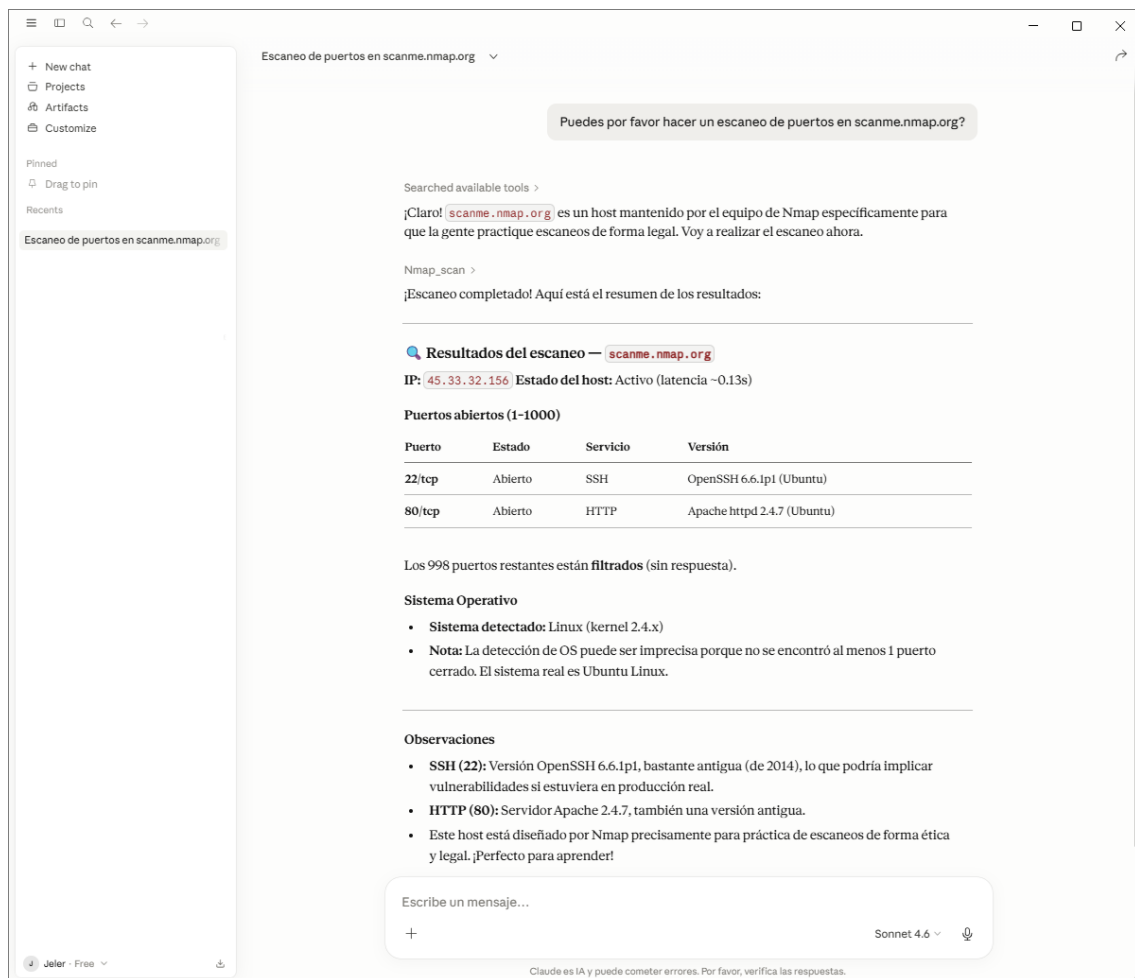
3. Pruebas

1. En Claude Desktop escribe:

¿Podrías hacer un escaneo de puertos básico con nmap en `scanme.nmap.org`?

2. Claude te pedirá **confirmación** antes de ejecutar el comando en Kali (haz clic en “Permitir”).

3. Revisa los logs en Kali (en la terminal donde corre kali-server-mcp) para ver la actividad.



Si funciona: ¡ya tienes Claude controlando Kali vía MCP! Puedes pedirle escaneos, explotación, wordlists, etc.

Anexo1

```
527 # Health check endpoint
528 @app.route("/health", methods=["GET"])
529 def health_check():
530     """Health check endpoint."""
531     # Check if essential tools are installed
532     essential_tools = ["nmap", "gobuster", "dirb", "nikto"]
533     tools_status = {}
534
535     for tool in essential_tools:
536         try:
537             result = execute_command(["which", tool])
538             tools_status[tool] = result["success"]
539         except:
540             tools_status[tool] = False
541
542     all_essential_tools_available = all(tools_status.values())
543
```

```
528 # Health check endpoint
529 @app.route("/health", methods=["GET"])
530 def health_check():
531     """Health check endpoint."""
532     # Check if essential tools are installed
533     essential_tools = ["nmap", "gobuster", "dirb", "nikto"]
534     tools_status = {}
535
536     for tool in essential_tools:
537         tools_status[tool] = shutil.which(tool) is not None
538
539     all_essential_tools_available = all(tools_status.values())
540
```